

MANUAL for DFTBopt

Version 1.0

Van Quan Vuong

March 17, 2026

Contents

1	An Introduction to DFTB Parameterization	3
1.1	DFTB	3
1.2	Electronic Parameters	4
1.3	Repulsive Potentials	5
1.4	Scoring Function	5
1.5	Genetic Algorithm	6
2	Overview and Installation of DFTBopt	7
2.1	Overview	7
2.2	Installation	7
3	Manual for REPOPT	9
3.1	Required Inputs	9
3.1.1	\$system:	9
3.1.2	\$genetic_algorithm:	10
3.2	Optional Inputs	12
3.2.1	\$element_types:	12
3.2.2	\$repulsive_potentials:	12
3.2.3	\$compounds:	13
3.2.4	\$definition_reactions:	14
3.2.5	\$reactions:	15
3.3	Output	15
3.4	Tips	16
4	Manual for EREPOPT	17
4.1	Overview	17
4.2	Input	18
4.2.1	\$system:	18
4.2.2	\$genetic_algorithm:	20
4.2.3	\$variables: (optional)	21
4.2.4	\$element_types:	22
4.2.5	\$atomparameters: (required for SK generation)	22
4.2.6	\$onecenterparameters: (required for SK generation)	23
4.2.7	\$twocenterparameters: (required for SK generation)	23
4.2.8	\$addhamiltonian: (optional)	23
4.2.9	\$atomic_energy: (optional; used in fit_type=1)	23

4.2.10	\$d3: (optional)	23
4.2.11	\$damph: (optional)	24
4.2.12	\$hubbardderivs: and \$hubbardderivsfull: (optional)	24
4.2.13	\$damphver2: (optional)	24
4.2.14	\$hubbards: (optional)	24
4.2.15	\$vorbes: (optional)	24
4.2.16	\$compounds: (required for fit_type=1/2)	24
4.3	Output	25
4.4	Tips	25
5	Utility Tools	26
5.1	Convert repopt -output to skf-files	26
5.2	Plot Repulsive Potentials	27
6	Tutorials	28
	Index	29

Chapter 1

An Introduction to DFTB Parameterization

1.1 DFTB

Expansion from DFT

$$\begin{aligned}
 E[\rho^0(r) + \Delta\rho(r)] = & \sum_i^{occ} \langle \psi_i | H^0 | \psi_i \rangle \Big\} E^{BS} \\
 & - \frac{1}{2} \int \int \frac{\rho^0(r') \rho^0(r)}{|r-r'|} dr' dr - \int v^{xc}[\rho^0(r)] \rho^0(r) dr \Big\} \approx \frac{1}{2} \sum_{a,b} V_{ab}^{rep}(R_{ab}) = E^{rep} \\
 & + E^{xc}[\rho^0(r)] + E^{NN} \\
 & + \frac{1}{2} \int \int \left(\frac{1}{|r-r'|} + \frac{\delta^2 E^{xc}[\rho(r)]}{\delta \rho(r') \delta \rho(r)} \Big|_{\rho^0(r') \rho^0(r)} \right) \Delta \rho(r') \Delta \rho(r) dr' dr \Big\} E^{2nd} \\
 & + \frac{1}{6} \int \int \int \left(\frac{\delta^3 E^{xc}[\rho(r)]}{\delta \rho(r'') \delta \rho(r') \delta \rho(r)} \Big|_{\rho^0(r'') \rho^0(r') \rho^0(r)} \right) \Delta \rho(r'') \Delta \rho(r') \Delta \rho(r) dr'' dr' dr \Big\} E^{3rd} \\
 & + \dots
 \end{aligned} \tag{1.1}$$

None Consistent-Charge (NCC)-DFTB

$$E^{NCC-DFTB} = \sum_i^{occ} \langle \psi_i | H^0 | \psi_i \rangle + E^{rep}. \tag{1.2}$$

Eigenvalue problem:

$$\sum_v^{AO} c_{vi} (H_{\mu v}^0 - \epsilon_i S_{\mu v}) = 0. \tag{1.3}$$

Hamiltonian matrix elements, $H_{\mu v}^0$:

$$H_{\mu v}^0 = \begin{cases} \langle \phi_\mu | -\frac{1}{2} \nabla^2 + V[\rho_a^0 + \rho_b^0] | \phi_v \rangle & \text{if } a \neq b \\ \epsilon^{\text{free atom}} & \text{if } a = b, \mu = v \\ 0 & \text{if } a = b, \mu \neq v. \end{cases} \tag{1.4}$$

Self-Consistent-Charge (SCC)-DFTB

$$E^{SCC-DFTB} = \sum_i^{occ} \langle \psi_i | H^0 | \psi_i \rangle + E^{rep} + \frac{1}{2} \sum_{a,b} \gamma_{ab}(R_{ab}) \Delta q_a \Delta q_b. \quad (1.5)$$

Eigenvalue problem:

$$\sum_{\nu}^{AO} c_{\nu i} (H_{\mu\nu} - \epsilon_i S_{\mu\nu}) = 0, \quad (1.6)$$

Hamiltonian, $H_{\mu\nu}$:

$$H_{\mu\nu} = H_{\mu\nu}^0 + \frac{1}{2} S_{\mu\nu} \sum_c (\gamma_{ac} + \gamma_{bc}) \Delta q_c. \quad (1.7)$$

Mulliken charge, Δq :

$$\Delta q_a = \frac{1}{2} \sum_i n_i \sum_{\mu \in a} \sum_{\nu} (c_{\mu i} c_{\nu i} S_{\mu\nu} + c_{\nu i} c_{\mu i} S_{\nu\mu}) - q_a^0, \quad (1.8)$$

Δq_c depends on MO coefficients $\Rightarrow$ must be solved iteratively.

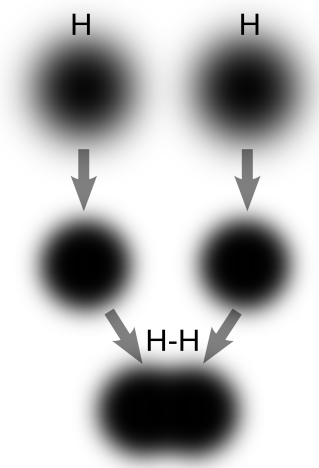
1.2 Electronic Parameters

- Minimal basis set:

Pure atomic orbitals (AOs) are too diffuse

- Electron density ρ^o :

ρ is more compressed in molecule



$$\Rightarrow \left[-\frac{1}{2} \nabla^2 + v^{eff}[\rho^{atom}] + \left(\frac{r}{r_o} \right)^2 \right] \phi_\mu = \epsilon_\mu \phi_\mu \quad (1.9)$$

Free variables:

Minimal AO basis set $\Rightarrow r_o^{wf}$

Electron density $\rho^o \Rightarrow r_o^{dens}$

1.3 Repulsive Potentials

Repulsive Potentials E^{rep} : sum of two-center repulsions,

$$E^{rep} = \frac{1}{2} \sum_{A,B} V_{AB}(|\mathbf{R}_A - \mathbf{R}_B|), \quad (1.10)$$

Where,

$$V_{AB}(R_{AB}) = \begin{cases} e^{-a_1 * R_{AB} + a_2} + a_3, & R_{AB} < R_{AB,0}, \\ \sum_{i=0}^4 a_{AB,n,i} (R_{AB} - R_{AB,n})^i, & R_{AB,n} \leq R_{AB} < R_{AB,n+1}; 4 \leq n \leq 6 \\ 0, & R_{AB,cut-off} \leq R_{AB}, \end{cases} \quad (1.11)$$

Free variables: $R_{AB,n}$ and $a_{AB,n,i}$.

1.4 Scoring Function

$$\begin{aligned} f^{score} = & \sum_{i \in equi} W_{at,i} |E_{at,i}^{ref} - E_{at,i}^{DFTB}| + \sum_{i \in bar} W_{bar,i} |E_{bar,i}^{ref} - E_{bar,i}^{DFTB}| \\ & + \sum_{i \in equi} W_{f,i} \sum_{j \in 3N_i} |F_{i,j}^{DFTB}| + \sum_{i \in pert} W_{f,i} \sum_{j \in 3N_i} |F_{i,j}^{ref} - F_{i,j}^{DFTB}|, \end{aligned} \quad (1.12)$$

$W_{at,i}, W_{bar,i}, W_{f,i}$: Weight factors

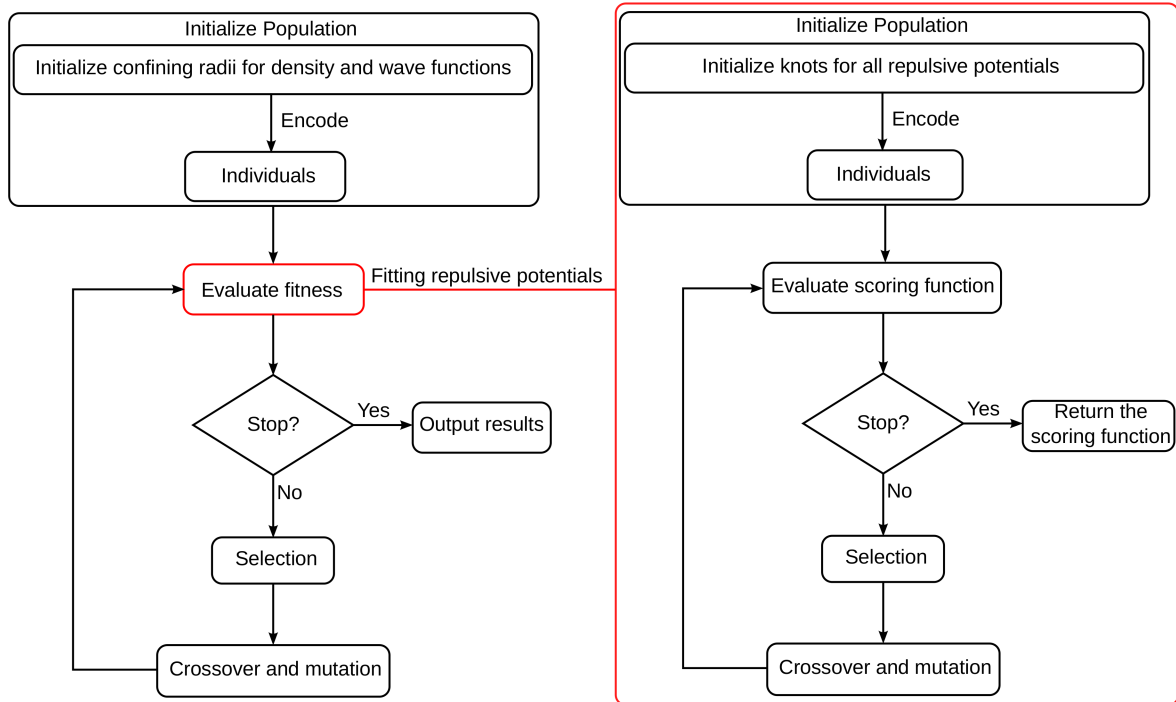
E_{at} : Atomization energies

E_{bar} : Proton transfer barriers

N : Number of atoms

F : Forces

1.5 Genetic Algorithm



Chapter 2

Overview and Installation of DFTBopt

2.1 Overview

DFTBopt[2] is a package to automatically optimize electronic, repulsive, and dispersion parameters for Density-Functional based Tight-Binding (DFTB) method. The package includes two main programs: (1) **repopt** for optimization of only repulsive potentials and (2) **erepopt** for optimization of all DFTB parameters. **erepopt** uses **repopt** for the repulsive potentials fitting. Currently, **repopt** is stable released version 1.0. On the other hand, **erepopt** is still in the beta-development version. In addition, the package also provide some tools to analyze or evaluate new DFTB parameters.

Most of the codes were written in C++. The code required two libraries: [galib247](#) and [eigen3](#). A part of the repulsive fitting (**repopt**) uses some piece of code from a semi-automatic erepfit program originally developed by Michael Gaus.[1] This manual was prepared using same style of DFTB+ manual, making it looks similar to DFTB+ manual. ☺☺☺

2.2 Installation

To get the code:

git clone <https://github.com/v2quan89/dftbopt.git>

or download compressed file

wget <https://github.com/v2quan89/DFTBopt.tar.gz>

Note, both links may not work at this moment. You can send an email to v2quan89@gmail.com if the links do not work. Then, extract DFTBopt.tar.gz using

```
tar -xvzf DFTBopt.tar.gz
```

Change to directory **DFTBopt** and type “./install.sh”. Currently, **DFTBopt** was tested for two compilers: GNU(g++) and INTEL(icpc).

You can choose the compiler by setting **\$CXX=“g++”** or **\$CXX=“icpc”** in the “install.sh” file. To compile “**erepopt**”, a MPI library is also required. The **install.sh** script will try to get all the required library and compile the code. You might have to adjust some flags in the **makefile**.

After the compilation, if you use **bash**, you can add following command to you “.bashrc” or run it before using DFTBopt,

```
source path-to-DFTBopt-directory/dftbopt_on.bash
```

Chapter 3

Manual for REPOPT

“**repopt**” is a program to optimize DFTB repulsive potentials. To run the program, type:

```
repopt rep.inp
```

where **repopt** is program name and **rep.inp** is name of the input file. The input file is organized in “block” sections. Each section begins with “**\$blockname:**” and end with “**\$end:**”. ‘#’ is the comment character. Everything after ‘#’ will be skipped.

3.1 Required Inputs

The general “block” format of required sections is showed as following

```
$blockname:
keyword_1          value_1
...
keyword_n          value_n
$end
```

Following input block must be presented in all kind of running job.

3.1.1 \$system:

Keyword	Type	Range	Default
dftb_version	string		dftb+
idecompose	integer	1:7	6
ilmsfit	integer	1:4	4
nreplicate	integer	≥ 1	1

dftb_version Executable dftb-program

idecompose Select decomposition method from EIGEN library:

1 => ldlt

2 => partialPivLu
 3 => fullPivLu
 4 => householderQr
 5 => colPivHouseholderQr
 6 => fullPivHouseholderQr
 7 => completeOrthogonalDecomposition
 Please check the [website](#) for more information.

ilmsfit Select regression method from EIGEN library:

1 => householderQr
 2 => colPivHouseholderQr
 3 => fullPivHouseholderQr
 4 => bdcSvd
 Please check the [website](#) for more information.

nreplicate Number of the fitting will be replicated. **nreplicate** should be 1. Larger than 1 only for testing purpose to measure the effect of decomposition and regression methods on the computing time.

Example:

```
$system:
  dftb_version      dftb+
  idecompose        6
  ilmsfit           4
  nreplicate        1
$end:
```

3.1.2 \$genetic_algorithm:

Keyword	Type	Range	Default
ga	bool	0 1	1
runtest	bool	0 1	0
score_type	integer	1 2 4	2
read_spline	bool	0 1	1
popsizemax	integer	≥ 1	1000
preserved_num	integer	≥ 0	100
destroy_num	integer	≥ 0	10
popsizemin	integer	≥ 1	2
ngen	integer	≥ 0	1000
pmut	integer	0.0:1.0	0.02
pcross	integer	0.0:1.0	0.90
grid_update	bool	0 1	0

ga switch on or off genetic algorithm

runtest switch on or off testing job

score_type set scoring function to:

1 => sum of absolute deviation

2 => sum of squared deviation

4 => sum of quartic deviation

read_spline how “grid” input file would be used:

0 => only read the cutoff and count the number of knots from “grid” input file

1 => use all knots in the “grid” input file as initial guess

popsizemax set the initial and maximum population size for GA.

preserved_num set the number of best individuals would be kept from “n-1” to “n” generation.

destroy_num set the number individuals be removed every generation. If greater than 0, the population size will be reduced generation by generation.

popsizemin set the final and minimum population size for GA. **popsizemin** is used only if **destroy_num** ≥ 1 .

ngen set the number of generation for GA.

pmut set the mutation probability for GA.

pcross set the crossover probability for GA.

grid_update How “grid” input file would be updated:

0 => leave the “grid” input file untouched.

1 => the “grid” input file is updated at the end of the GA optimization using the best found knots.

Example:

```
$genetic_algorithm:
    ga                      1
    runtest                 0
    score_type              2
    read_spline             1
    popsizemax              1000
    preserved_num           100
    destroy_num             10
    popsizemin              2
    ngen                    1000
    pmut                    0.02
    pcross                  0.90
    grid_update             0
$end:
```

3.2 Optional Inputs

The general “block” format of required sections is showed as following

\$blockname:			
entry_name_1	option_1_1	...	option_1_m
...	...	...	...
entry_name_n	option_n_1	...	option_n_m
\$end			

Following input blocks in optional, depending on the desired job.

3.2.1 \$element_types:

String	Real(a.u.)
element_name_1	atomic_energy_1
...	...
element_name_n	atomic_energy_n

element_name name of fitting element, must be two lower case letters characters long. The underscore character ‘_’ is added if element name has only one character.

atomic_energy Atomic energy in a.u. for the corresponding fitting element.

Note: if **element_name** is provided, **atomic_energy** must be provided also. **atomic_energy** (can be calculated by DFT) is needed to fit atomization energy. If **element_name** is not listed, the **atomic_energy** will be optimized. You can interpret the meaning of **atomic_energy** as: If **atomic_energy** is provided, the absolute atomization energy will be fitted (by fitting atomization energy). If **atomic_energy** is optimized (not provided), the relative atomization energy (reaction energy) will be fitted (by optimization of the atomic energy). This methodology was proposed by Gaus et al. in the 3ob parameterization strategy for obtaining repulsive potentials, and their optimized atomic energies (used to generate the 3ob repulsives) were published in their supporting information.[?]

Example:

\$element_types:	
h_	-0.256789
c_	-0.456789
\$end:	

3.2.2 \$repulsive_potentials:

string	Real(Å)	string	Real(Å)	integer	integer	0 1
name_1	min_r	knot-vector	min_step	spline order	smooth	negative?
...	...	...	...	...	...	...
name_n	min_r	knot-vector	min_step	spline order	smooth	negative?

name name of fitting potential

min_r limit the small knot. The small knot must larger than or equal to shortest bond length $\min(R_{bond}) - \min_r$

knot-vector name of the file containing division points in the format (in Å):

knot_1

...

knot_n

cutoff

The number of knot will be counted from the not-vector

min_step set the smallest difference between knot

spline_order the order of spline function to be used (currently only support 4th order)

smooth smoothing level of that potential

0 => constrain on potential energy

1 => constrain on the first derivative of energy

2 => constrain on the second derivative of energy

3 => constrain on the third derivative of energy

negative? and allowance the potential to be attractive or not.

0 => repulsive potential energy must be always positive 1 => repulsive potential energy can be negative

Example:

```
$repulsive_potentials:
  h_h_      0.2      grids/hh.grdx      0.05      4      2      0
  c_h_      0.3      grids/ch.grdx      0.05      4      2      1
  c_c_      0.3      grids/cc.grdx      0.30      4      2      1
$end
```

3.2.3 \$compounds:

string	Real(kcal/mol)	string	Real	string	0 string	integer
structure1	E^{at}	eweight	fweight	dftbinp	forceinput	placeholder
...	...	...	...	...	...	...
structuren	E^{at}	eweight	fweight	dftbinp	forceinput	placeholder

list of filenames for geometries of the fitting molecular.

name file name for geometry, the files need to be in xyz-format

E^{at} reference atomization energy of the molecule. The atomization energy is defined as:

$$E^{at} = -E^{tot} + \sum_{i=1}^{N_{atom}} E_i^{atom}$$

eweight weights for energy equations

fweight weights for force equations

dftbinp input-file to run a single point energy and force calculation using the dftb

forceinput for an equilibrium structure, should be a “0”, otherwise a reference force file can be specified which is formatted as (in a.u.):

```
Ref_Force-Atom_1_X Ref_Force-Atom_1_Y Ref_Force-Atom_1_Z
...
Ref_Force-Atom_n_X Ref_Force-Atom_n_Y Ref_Force-Atom_n_Z
```

placeholder for development only, must be ‘0’ for now

Example:

```
$compounds:
  path/h2.xyz      109.9    1    1    path/dftb_inp1.hsd    0          0
  path/ch4.xyz     420.1    1    1    path/dftb_inp2.hsd    0          0
  path/h3cch3.xyz  712.0    1    1    path/dftb_inp2.hsd    0          0
  path/h2_d0.1.xyz  000.0    0    1    path/dftb_inp2.hsd    path/hh_d0.1.frc    0
$end
```

3.2.4 \$definition_reactions:

For specifying reaction equations

string	string	string
abbreviation	filename	dftbinp
...	...	...
abbreviation	filename	dftbinp

abbreviation abbrev name for a geometry

filename file name for the geometry, the files need to be in xyz-format

dftbinp input-file to run a single point energy calculation using the DFTB

Example:

\$definition_reactions:		
h2	path/h2.xyz	path/dftb_inp1.hsd
ch4	path/ch4.xyz	path/dftb_inp2.hsd
h3cch3	path/h3cch3.xyz	path/dftb_inp2.hsd
\$end		

3.2.5 \$reactions:

integer	string		integer	string	string	Real(kcal/mol)	Real
coeff	abbreviation	...	coeff	abbreviation	->	reactionenergy	reaweight
...	...	...	...	...	->	...	...
coeff	abbreviation	...	coeff	abbreviation	->	reactionenergy	reaweight

coeff reaction coefficient
 if positive => reactant
 if negative => product

abbreviation defined in the \$definition_reactions: block

reactionenergy reaction energy

reaweight weight for reaction energy equations

Example:

\$reactions:								
+1	h3cch3	+1	h2	-2	ch4	->	-18.33	1.0
\$end								

3.3 Output

The output of a successful **repopt** contains:

scoring function scoring function as a function of generation

input interpreted input, a list of all distances appearing within the reference geometries sorted by atom type pair.

technical information a list of number of fitting equation, number of free variables...

summary of fitting summary of the MSE, MUE, and RMS

residual in detail residuals for each equation predicted by the fitted parameters in comparison to the reference are listed.

fitted atomic energies fitted atomic energies if they are optimized

repulsive potentials the repulsive potentials are given in a format of the “Spline” format.

If there is no error, **repopt** ends with a statement “repopt normal termination”. Any warnings concerning the fit will appear after **#ga end!**.

3.4 Tips

Chapter 4

Manual for EREPOPT

“**erepopt**” is a program to optimize all DFTB parameters simultaneously. To run the program, type:

```
erepopt erep.inp
```

where **erepopt** is program name and **erep.inp** is name of the input file. The input file is organized in “block” sections. Each section begins with “**\$blockname:**” and end with “**\$end:**”. ‘#’ is the comment character. Everything after ‘#’ will be skipped.

4.1 Overview

erepopt couples a genetic algorithm (GA) optimizer with external DFTB parameter generation and scoring tools. In each GA step, a candidate parameter set is translated into DFTB Slater–Koster (.skf) files (via skgen and optional post-processing), then evaluated with one of the scoring modes selected by **fit_type**:

fit_type = 0 Fit repulsive potentials by calling **repopt** internally. This is the default mode and requires a valid **rep.in** input for **repopt**.

fit_type = 1 Fit band energies / eigenvalues for each compound against a reference list.

fit_type = 2 Fit density of states (DOS) curves against reference DOS data. This mode runs **dftb+** and **dp_dos**.

Parallelization. **erepopt** is compiled with MPI and typically run with multiple ranks. Inside each rank it can also use OpenMP for external tools via **nthreads** (exported as OMP_NUM_THREADS).

4.2 Input

4.2.1 \$system:

Keyword	Type	Range	Default
nthreads	integer	≥ 1	1
dftbversion	string		dftb+
skgen	string		skgen
onecent	string		hfatom_spin
twocent	string		sktwocnt_lr
skdefversion	integer		(no default)
xcfunctional	string		(no default)
omega	Real		(unset)
superposition	string		(no default)
repopt	string		repopt
power	integer	≥ 2	2
dgrid	Real	≥ 0.0	0.1
ngrid	integer	≥ 1	100
grids	string		grids
rep.in	string		rep4e.in
libdir	string		libskf4e
libaddir	string		(unset)
repaddir	string		(unset)
scratchfolder	string		/tmp
skfclean	bool	0 1	1
outfile	string		erepopt.log
popinitialfile	string		pop.initial.dat
popfinalfile	string		pop.final.dat

nthreads Number of OpenMP threads used for external calls made by **erepopt** (e.g. skgen and **repopt**). This value is exported as OMP_NUM_THREADS.

dftbversion DFTB executable used in **fit_type = 1/2** (default: dftb+).

skgen The skgen executable.

onecent The one-center executable used by skgen (default: hfatom_spin).

twocent The two-center executable used by skgen (default: sktwocnt_lr).

skdefversion Version number written to SkdefVersion in the generated skdef.hsd.

xcfunctional XC functional string written to XCFunctional in skdef.hsd when omega is not used.

omega If provided, enables long-range corrected mode and writes func = xchybrid { omega = ... } and LC omega markers in .skf files.

superposition Value written to Superposition in skdef.hsd.

repopt Path to **repopt** executable (used in **fit_type = 0**).

power Power used in skdef.hsd (PowerCompression). See \$atomparameters:.

dgrid Grid spacing parameter (default: 0.1). In the current **erepopt** code this value is not passed to **repopt**; it is only used as an internal setting and may be relevant for auxiliary tools or future extensions.

ngrid Integer threshold used by an internal SKF sanity check (default: 100). When validating an existing .skf, **erepopt** can require that the Spline marker appears after at least ngrid lines. (Note: the current code primarily checks for the presence of Spline and, when omega is enabled, a matching LC omega; the grid-length check is not enforced in the default pass criteria.)

grids Path to a grid file or directory copied into the rank-local scratch working directory before calling **repopt** (**fit_type = 0**). This is the only one of the three parameters that is directly used to stage input for **repopt**.

rep.in Repopt input file. In **fit_type = 0** this file is read and a rank-local rep.in is created in scratch with optional additional blocks (\$d3:, \$damph:, \$hubbardderivs:, \$damphver2:).

libdir Output directory used to store generated .skf files. **erepopt** reads from and writes to this directory during optimization.

libadddir Optional directory containing additional .skf files that are copied into the working SKF directory for scoring (e.g. element pairs you do not want to optimize). If set, all *.skf in this directory are copied for each evaluation.

repadddir Optional directory containing repulsive (Spline) tails appended to generated .skf files.

scratchfolder Base scratch directory. Each MPI rank uses subfolders slakos.tmp_RANK, buildsks.tmp_RANK, and (for **fit_type = 2**) dftb.tmp_RANK.

skfclean If set to 1, **erepopt** deletes all files in libdir after each generation (this reduces disk usage but can remove intermediate results).

outfile Main log file written by rank 0 (default: gaserepfit.log).

popinitialfile, **popfinalfile** Population snapshots written by rank 0. Each line is an individual: all genes (radii + optional \$d3:/\$damph:/\$hubbardderivs:/\$damphver2:/\$hubbards:/\$vorbes:) followed by the individual score.

How SKF files are reused and when repulsive tails are appended. During each evaluation, **erepopt** ensures that all required element-pair SKF files exist. For an element pair X, Y , it considers both directions ($X-Y.skf$ and $Y-X.skf$). The logic implemented in the current code is:

- **Prefer existing SKFs from libadddir (skip generation).** If libadddir is set and a file $X-Y.skf$ exists there, the pair is treated as available and **skgen** is **not called** for that pair. (The same applies symmetrically for $Y-X.skf$.)
- **Otherwise, check libdir for already-generated SKFs.** If either $X-Y.skf$ or $Y-X.skf$ exists in libdir and passes an internal sanity check (presence of required markers such as Spline, and when omega is enabled also matching LC omega), the pair is treated as available and **skgen** is **not called**.

- **Only if both directions are missing/invalid, call skgen once to generate both.** When both X-Y.skf and Y-X.skf are missing (or fail the sanity check), **erepopt** runs `skgen ... sktable X Y`. This single call produces both direction files, which are then moved into `libdir`.
- **Repulsive tails from repadddir are appended only on the generation path.** If `repadddir` is set and a matching tail file exists (e.g. `repadddir/X-Y.skf`), it is appended to the newly generated SKF file(s) right after `skgen` finishes (in the rank-local scratch directory) and the resulting SKF files are then moved into `libdir`. Therefore, **SKF files written to libdir by erepopt already include the appended repulsive tails** when `repadddir` is enabled and the tail files exist. If generation is skipped because an SKF already exists in `libdir` or `libadddir`, **no additional repulsive-tail post-processing is performed** on that existing file.

Example:

```
$system:
  nthreads          1
  dftbversion       dftb+
  skgen             skgen
  onecent           hfatom_spin
  twocent           sktwocnt_lr
  repopt            repopt
  power             2
  dgrid             0.1
  ngrid             120
  grids             grids
  rep.in            rep4e.in
  libdir            libskf4e
  scratchfolder     /dev/shm
  skfclean          0
  outfile           gaserepfit.log
  popinitialfile    pop.initial.dat
  popfinalfile      pop.final.dat
$end
```

4.2.2 \$genetic_algorithm:

Keyword	Type	Range	Default
ga	bool	0 1	1
runtest	bool	0 1	0
fit_type	integer	0 1 2	0
popsize	integer	≥ 1	1000
preserved_num	integer	≥ 0	100
ngen	integer	≥ 0	1000
pmut	integer	0.0:1.0	0.02
pcross	integer	0.0:1.0	0.90
readr	bool	0 1	1
restart	bool	0 1	1

ga Switch GA on/off. In the current implementation, **erepopt** performs optimization only when `ga=1` and `runtest=0`.

runtest If 1, run in testing mode (no GA optimization).

fit_type Select scoring mode: 0 = call **repopt** (repulsive fitting); 1 = eigenvalue fitting; 2 = DOS fitting.

popsize Population size.

preserved_num Number of elite individuals preserved from generation $n - 1$ to n .

ngen Number of generations.

pmut Mutation probability.

pcross Crossover probability.

readr If 1, the first initialization uses the values given in `$element_types:` and optional blocks (`$d3:`, `$damph:`, etc.) as the initial genome; otherwise random initialization is used.

restart If 1, read `popfinalfile` and use its gene values as starting points.

Example:

\$genetic_algorithm:	
ga	1
runtest	0
fit_type	0
popsiz	32
preserved_num	3
ngen	30
pmut	0.05
pcross	0.9
readr	1
restart	0
\$end:	

4.2.3 \$variables: (optional)

This block controls internal coupling rules used to constrain genes for `$element_types:` via the `optype` field. These variables are mainly intended for advanced users and development workflows.

Keyword	Type	Range	Default
s1	Real		1.17
s2	Real		1.40
s3	Real		1.20
s4	Real		2.00
s5	Real		2.00
s6	Real		2.00
s7	Real		2.00
s8	Real		2.00
s9	Real		2.00
sdenswf	Real		2.00
idrefr	integer	≥ 0	3

Note: idrefr is an index into the flattened genome vector (0-based in code) used as a reference gene by some optype coupling schemes.

4.2.4 \$element_types:

This block defines the list of chemical elements to be optimized, together with their radii gene bounds, initial values, and discretization precision. Each line has the form:

ElementName optype lmax (min r max prec) $_{lmax+2}$

ElementName Element symbol used by skgen (e.g. H, C, O). Case sensitive.

optype Integer controlling coupling constraints among radii genes (advanced usage; see \$variables:).

lmax Maximum angular momentum channel. The number of radii genes for this element is $lmax + 2$: one density-compression radius (index 0) and $(lmax + 1)$ wave-compression radii (S, P, D, ...).

min, r, max Minimum / initial / maximum value for each radius gene.

prec Precision level, used to set the mutation step size δ : $0 \rightarrow 1.0$, $1 \rightarrow 0.1$, $2 \rightarrow 0.01$, $3 \rightarrow 0.001$, $4 \rightarrow 0.0001$, $5 \rightarrow 0.00001$.

Example:

\$element_types:									
H	11	0	2.9 2.9 2.9	1	2.9 2.9 2.9	1			
O	111	1	2.7 2.8 2.9	1	2.7 2.8 2.9	1	2.7 2.8 2.9	1	
N	111	1	3.0 3.2 3.4	1	3.0 3.2 3.4	1	3.0 3.2 3.4	1	
C	111	1	3.6 3.8 4.0	1	3.6 3.8 4.0	1	3.6 3.8 4.0	1	
\$end									

4.2.5 \$atomparameters: (required for SK generation)

This block provides per-element AtomParameters entries for skdef.hsd. Each element starts with the element name on its own line, followed by a sub-block terminated by \$subend. Supported keywords are:

PowerCompression Integer power used in PowerCompression.

ShellResolved Value written to ShellResolved.

All other lines inside the element sub-block are copied verbatim into skdef.hsd.

4.2.6 \$onecenterparameters: (required for SK generation)

Lines inside this block are copied verbatim into the OnecenterParameters section of skdef.hsd.

4.2.7 \$twocenterparameters: (required for SK generation)

Lines inside this block are copied verbatim into the TwoCenterParameters section of skdef.hsd.

4.2.8 \$addhamiltonian: (optional)

Lines inside this block are appended verbatim inside the Hamiltonian = DFTB { ... } section of the rank-local dftb_in.hsd file used for **fit_type = 1/2**. This is the primary way to add DFTB+ Hamiltonian options (e.g. SCC parameters, dispersion, third-order terms, etc.).

4.2.9 \$atomic_energy: (optional; used in fit_type=1)

This block provides per-element atomic energy values used to compute a baseline energy b0 for each compound in **fit_type = 1**. Each line is:

element_name atomic_energy

4.2.10 \$d3: (optional)

This block defines D3-related scalar parameters optimized together with radii. Each line is:

name min value max precision

Example:

\$d3:				
s6	1.00	1.00	1.00	2
s8	1.40	1.40	1.40	1
a1	0.48	0.48	0.48	2
a2	4.70	4.70	4.70	1
\$end:				

4.2.11 \$damph: (optional)

Damping parameter optimized as a single scalar. The line format is:

min value max precision

4.2.12 \$hubbardderivs: and \$hubbardderivsfull: (optional)

Hubbard derivative parameters optimized as per-element scalars. Each line is:

name min value max precision

Note: \$hubbardderivsfull: enables a different internal flag but uses the same line format.

4.2.13 \$damphver2: (optional)

Second version of damping parameters, optimized as per-element scalars. Each line is:

name min value max precision

4.2.14 \$hubbards: (optional)

Customized Hubbard parameters per element. Each line is:

name type min value max precision

4.2.15 \$vorbes: (optional)

Customized on-site parameters per element. Each line is:

name type min value max precision

Example:

\$vorbes:						
N	2S	-0.83	-0.82	-0.81		3
\$end:						

4.2.16 \$compounds: (required for fit_type=1/2)

This block defines the training set. Each line is:

xyzfile charge start end gauss reference__data weight

xyzfile Geometry in XYZ format. Element names in the XYZ are lowercased internally.

charge Total charge passed to DFTB+.

start, end Energy window (in the same units as the reference data) used for scoring.

gauss Gaussian broadening used by `dp_dos` (`fit_type=2`) and stored for logging.

reference_data Reference file: in **fit_type = 1** it is read as a list of eigenvalues and occupations; in **fit_type = 2** it is read as a DOS curve (energy, DOS).

weight Weighting factor. For **fit_type = 2**, special negative values implement energy-dependent weighting: -1 linear, -2 quadratic, -4 quartic weighting from start to end.

4.3 Output

erepopt writes several outputs during a GA run:

Main log (outfile) Written by rank 0. It contains progress lines during evolution and ends with “***** erepobj normal termination *****” on success. After GA completion it prints an “interpreted input” summary, the final optimized parameters, and per-compound errors.

score.dat GA score history written by GALib (frequency controlled by `scoref/flushf`).

popinitialfile, popfinalfile Population snapshots written by rank 0 at initialization and after every generation.

Generated SKF files Stored in `libdir`. File names encode the element radii and optional parameter blocks, e.g. `X_d..._w...-Y_d..._w..._hub..._vor....skf` (and an additional `omega_...` prefix if `omega` is set).

Temporary scratch Rank-local folders under `scratchfolder` are created and deleted during the run.

4.4 Tips

Run under a scheduler Since **erepopt** uses MPI, on clusters you typically run it with `srun` (recommended) or `mpirun`. Example (2 MPI ranks, 8 threads each):

```
srun -n 2 -c 8 erepopt erep.inp
```

Validate tool paths first Ensure `skgen`, `onecent`, `twocent`, `repopt`, `dftbversion`, and (for **fit_type = 2**) `dp_dos` are available in `$PATH` or specified with absolute paths.

Start from a small training set Debug input formats and external tool calls with 1–2 compounds and a small `popsize/ngen` before scaling up.

Be careful with skfclean Setting `skfclean=1` removes all `.skf` files in `libdir` each generation, which can make debugging difficult. Consider using `skfclean=0` while developing.

Common failure modes Most runtime errors come from missing files or failing external calls. If you see an error mentioning `rep.out` or missing DOS output, inspect the corresponding `scratch` directory for the rank and check the external program logs (`rep.out`, `log`, `dos.dat`).

Chapter 5

Utility Tools

In the following section, some utility tools will be explained. These tools were originally developed by Michael Gaus and were later modified by the author.

5.1 Convert repopt-output to skf-files

The bash-scripts **rep2XabSpl** and **xabSpl2spl** are available in the “utils” directory as well as the C++ program **ord2abSpl** which is called by the **xabSpl2spl** script.

rep2XabSpl Usage: **rep2XabSpl repout-output-file**

The **rep2XabSpl** extracts the Spline of repulsive potentials from the output-file and writes it in separate files. The ending of the files are XabSpl.

xabSpl2spl Usage: **xabSpl2spl XabSpl-file skf-electronic-file 1**

The **xabSpl2spl** script combines one XabSpl file with skf-electronic-file into the final skf-file.

ord2abSpl called by the **xabSpl2spl**

For a short description of all options run **rep2XabSpl** or **xabSpl2spl** without any arguments.

Example

```
# doing the rep fitting
repopt rep.in > rep.out
```

```
# extract Spline for H-H
rep2XabSpl rep.out
mv h_h_4abSpl hh.4abSpl
```

```
# create the final skf-files
xabSpl2spl hh.4abSpl hh_elec.skf 1
```

Under **utils** folder, a script named **combine.sh** can do all jobs at once.

5.2 Plot Repulsive Potentials

SplineAnsCh is a script to plot repulsive potentials and its derivatives. The script requires [gnuplot](#) and [gv](#) ghostscript interpreter.

Usage

one skf file: `SplineAnsCh -a  $r_{min}:r_{max}$  file1.skf`

two skf file: `SplineAnsCh -a  $r_{min}:r_{max}$  -v file1.skf file2.skf`

Note: any files in the XabSpl or spl format can be also be used. You can find all options by running SplineAnsCh without arguments.

Example

```
# to plot new cc.skf zoom in on a range of 2.0-5.0 (a.u.).  
SplineAnsCh -a 2.0:5.0 cc.skf
```

```
# to compare the new cc.skf with cc.skf from mio set.  
SplineAnsCh -a 2.0:5.0 -v cc_mio.spl cc.4abSpl
```

Chapter 6

Tutorials

For **repopt**, there are two examples rep1.in and rep2.in under the **examples** folder. It is straight forward to run these examples:

```
# cd to the examples folder and type  
repopt rep1.in >& rep1.out  
repopt rep2.in >& rep2.out
```

For **erepopt**, the exmamples are under construction.

Bibliography

- [1] Michael Gaus, Chien-Pin Chou, Henryk Witek, and Marcus Elstner. Automatized Parametrization of SCC-DFTB Repulsive Potentials: Application to Hydrocarbons. *J. Phys. Chem. A*, 113(43):11866–11881, 2009. [7](#)
- [2] Van Quan Vuong, Jissy Akkarapattiakal Kuriappan, Maximilian Kubillus, Julian J. Kranz, Thilo Mast, Thomas A Niehaus, Stephan Irle, and Marcus Elstner. Parametrization and Benchmark of Long-Range Corrected DFTB2 for Organic Molecules. *J. Chem. Theory Comput.*, 14(1):115–125, 2018. [7](#)